

Developer's Guide

Author: Raja SP, AWS Labs

Source: <https://github.com/awslabs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [developers-guide.pdf](#)

Running CodeBuild Locally

You can run AWS CodeBuild builds locally using the [CodeBuild local agent](#). This is useful for testing buildspec changes without pushing to the remote.

Prerequisites

- Docker installed and running
- The `codebuild_build.sh` script:

Basic Usage

1. Setup
2. Download the local CodeBuild script and make it executable.
3. Send the `GH_TOKEN` environmental GitHub Personal Access Token (PAT) into a `./.env` file

```
if [ ! -f codebuild_build.sh ]; then
  curl -O https://raw.githubusercontent.com/aws/aws-codebuild-docker-images/master/local_bu
lds/codebuild_build.sh && chmod +x codebuild_build.sh;
fi;
echo "GH_TOKEN=${GH_TOKEN:-ghp_notset}" > "./.env";
```

1. Iterate
2. Optionally edit the `buildspec-override` value in the `.github/workflows/codebuild.yml` GitHub workflow

3. Update `./buildspec.yml` based on the workflow contents to a local file
4. Run AWS CodeBuild build locally with images based on the machine architecture

```
cat .github/workflows/codebuild.yml \
    | uvx yq -r '.jobs.build.steps[] | select(.id == "codebuild") | .with["buildspec-overrid
e"]' \
    > buildspec.yml
./codebuild_build.sh \
    -i "public.ecr.aws/codebuild/amazonlinux-$([ "$(arch)" = "arm64" -o "$(arch)" = "aarch64"
] && echo "aarch64" || echo "x86_64")-standard:$([ "$(arch)" = "arm64" -o "$(arch)" = "aarch
64" ] && echo "3.0" || echo "5.0")" \
    -a "./.codebuild/artifacts/" \
    -l "public.ecr.aws/codebuild/local-builds:$([ "$(arch)" = "arm64" -o "$(arch)" = "aarch64"
] && echo "aarch64" || echo "latest")" \
    -c \
    -e "./.env"
```

All Script Options

Flag	Required	Description
<code>-i</code> <code>IMAGE</code>	Yes	Customer build container image (e.g. <code>aws/codebuild/standard:5.0</code>)
<code>-a DIR</code>	Yes	Artifact output directory
<code>-b FILE</code>	No	Buildspec override file. Defaults to <code>buildspec.yml</code> in the source directory
<code>-s DIR</code>	No	Source directory. First <code>-s</code> is the primary source; additional <code>-s</code> flags use <code><sourceIdentifier>:<sourceLocation></code> format for secondary sources. Defaults to the current working directory
<code>-l</code> <code>IMAGE</code>	No	Override the default local agent image
<code>-r DIR</code>	No	Report output directory
<code>-c</code>	No	Use AWS configuration and credentials from your local host (<code>~/.aws</code> and <code>AWS_*</code> environment variables)
<code>-p</code> <code>PROFILE</code>	No	AWS CLI profile to use (requires <code>-c</code>)
<code>-e FILE</code>	No	File containing environment variables (<code>VAR=VAL</code> format, one per line)
<code>-m</code>	No	Mount the source directory into the build container directly
<code>-d</code>	No	Run the build container in Docker privileged mode

Security Scanners

The `security-scanners.yml` workflow runs six scanners on every push to `main`, every PR targeting `main`, and on a daily schedule. Each scanner uploads a SARIF report to GitHub Code Scanning (visible under the **Security** tab) and as a downloadable artifact.

All scanners except ClamAV use a **deferred-failure pattern**: the scan always runs to completion and uploads results before the job fails. This ensures findings are recorded

even when the build breaks.

Bandit — Python SAST

What it detects: Common security issues in Python code (e.g., use of `subprocess`, `eval`, hardcoded passwords, weak crypto).

What triggers failure: Any finding with **high confidence**, at any severity level. See the Bandit configuration in `.github/workflows/security-scanners.yml` for the exact filters used.

Scope: Runs against all tracked Python files in the repository; see `.github/workflows/security-scanners.yml` for the precise include/exclude patterns.

How to review findings:

1. Check the **Code Scanning** alerts in the GitHub Security tab, or download the `bandit.sarif` artifact
2. Each finding includes a Bandit rule ID (e.g., `B603`) and a description of the risk

How to remediate:

- **Fix the code** — the preferred approach. Bandit docs list safe alternatives for each rule
- **Suppress inline** — add `# nosec BXXX` (with a justification) to the affected line:

```
python subprocess.run(cmd, check=True) # nosec B603 - cmd is built from validated config, not user input
```

- **Exclude a path** — add to the `exclude` list in `.bandit`

Semgrep — Multi-language SAST

What it detects: Security anti-patterns, dangerous API usage, and code quality issues across all languages using the full Semgrep Registry (`--config=r/all`).

What triggers failure: Any finding. On PRs, only **new** findings (vs the PR base commit) trigger failure — pre-existing findings are ignored via `--baseline-commit`.

How to review findings:

1. Check **Code Scanning** alerts or download the `semgrep.sarif` artifact

2. Each finding includes a rule ID (e.g., `python.lang.security.dangerous-subprocess-use-audit`) and a link to the rule documentation

How to remediate:

- **Fix the code** — follow the rule's suggested fix in the Semgrep Registry docs
- **Suppress inline** — add `# nosemgrep: <rule-id>` to the affected line:

```
python time.sleep(5) # nosemgrep: arbitrary-sleep - polling for server startup
```

For YAML files:

```
yaml run: exit ${ steps.scan.outputs.exit_code } # nosemgrep: yaml.github-actions.security.curl-eval.curl-eval
```

- **Exclude a path** — add the path to `.semgrepignore` (note: the `changed-semgrepignore` audit rule will flag new entries for app-sec review)

Grype — Dependency Vulnerability Scanning (SCA)

What it detects: Known CVEs in project dependencies by scanning lock files, manifests, and container images.

What triggers failure: Any vulnerability rated **high or critical** (`fail-on-severity: high` in `.grype.yaml`). Low and medium vulnerabilities are reported but do not fail the build.

How to review findings:

1. Check **Code Scanning** alerts or download the `grype.sarif` artifact
2. Each finding includes the CVE ID, affected package, installed version, and fixed version (if available)

How to remediate:

- **Upgrade the dependency** — the preferred approach. Check if a patched version exists and update the relevant `pyproject.toml` or lock file
- **Suppress in config** — add an entry to the `ignore` list in `.grype.yaml` with a reason:

```
yaml ignore: - vulnerability: CVE-2024-12345 reason: "only affects server-side XML parsing which we don't use"
```

You can scope to a specific package:

```
yaml ignore: - vulnerability: CVE-2024-12345 package: name: "some-package" version: "1.2.3"
reason: "pinned version; affected code path is unreachable"
```

Note: Gripe is an SCA scanner — it analyzes dependencies, not source lines. There are no inline code comments for suppression; all accepted risks go in `.gripe.yaml`.

Gitleaks — Secret Detection

What it detects: Secrets (API keys, tokens, passwords, private keys) committed anywhere in the git history.

What triggers failure: Any secret not present in the baseline file (`.gitleaks-baseline.json`).

How to review findings:

1. Download the `gitleaks.sarif` artifact
2. Each finding identifies the secret type (e.g., `generic-api-key`, `jwt`), file, and commit

How to remediate:

- **Rotate the secret immediately** — treat any detected secret as compromised
- **Remove from history** — use `git filter-repo` or BFG Repo-Cleaner to purge the secret from all commits
- **Add to baseline** — only for known false positives (e.g., test fixtures with synthetic credentials). Regenerate the baseline:

```
bash gitleaks git --config=.gitleaks.toml --report-path=.gitleaks-baseline.json --report-format=json .
```

Review the updated baseline carefully before committing

- **Allowlist a path** — add a regex to `.gitleaks.toml` under `[allowlist] paths` for files that intentionally contain secret-like patterns (e.g., test credential scrubbers)

Checkov — Infrastructure as Code Scanning

What it detects: Misconfigurations in GitHub Actions workflows and Dockerfiles (e.g., unpinned actions, missing security settings, overly broad permissions).

Scope: Only scans `github_actions` and `dockerfile` frameworks (configured in `.checkov.yaml`).

What triggers failure: Any check failure, except checks listed in `skip-check`.

How to review findings:

1. Check **Code Scanning** alerts or download the `checkov.sarif` artifact
2. Each finding includes a check ID (e.g., `CKV_GHA_7`, `CKV_DOCKER_2`) and a description of the misconfiguration

How to remediate:

- **Fix the configuration** — follow the Checkov docs for the specific check ID
- **Suppress inline** — add a comment above or on the affected line:

In a Dockerfile:

```
dockerfile # checkov:skip=CKV_DOCKER_2:healthcheck not needed for build-only image FROM  
python:3.12-slim
```

In a GitHub Actions workflow:

```
yaml # checkov:skip=CKV_GHA_7:buildspec-override requires user parameters - uses: aws-  
actions/aws-codebuild-run-build@v1
```

Multiple skips on one line:

```
yaml # checkov:skip=CKV_DOCKER_2,CKV_DOCKER_3:reason for both
```

- **Skip repo-wide** — add the check ID to the `skip-check` list in `.checkov.yaml` with a comment explaining why

ClamAV — Malware Scanning

What it detects: Malware, viruses, and trojans in repository files using ClamAV's signature database.

What triggers failure: Any malware detection (binary pass/fail).

How to review findings:

1. Download the `clamscan.txt` artifact — it contains the full scan log with any infected file paths

Note: ClamAV does not produce SARIF output and does not integrate with GitHub Code Scanning. Results are only available as the text log artifact.

How to remediate:

- **Remove the infected file** and investigate how it was introduced
- **Verify the detection** — false positives are rare but possible. Check the ClamAV signature name against known FP databases

Summary of Failure Thresholds

Scanner	Fails on	Severity filter	Config file
Bandit	Any finding with high confidence	All severities	<code>.bandit</code>
Semgrep	Any finding (PRs: new only)	All severities	<code>.semgrepignore</code>
Grype	High or critical CVEs	Low/medium don't fail	<code>.grype.yaml</code>
Gitleaks	Any secret not in baseline	All	<code>.gitleaks.toml</code> , <code>.gitleaks-baseline.json</code>
Checkov	Any check failure	All (minus skipped)	<code>.checkov.yaml</code>
ClamAV	Any malware detection	Binary pass/fail	None

Summary of Suppression Methods

Scanner	Inline comment	Config-level	Baseline/differential
Bandit	<code># nosec BXXX</code>	<code>.bandit</code> <code>exclude</code>	—
Semgrep	<code># nosemgrep: rule-id</code>	<code>.semgrepignore</code>	<code>--baseline-commit</code> on PRs
Grype	<i>(not applicable — SCA)</i>	<code>.grype.yaml</code> <code>ignore</code>	—
Gitleaks	—	<code>.gitleaks.toml</code> <code>allowlist</code>	<code>.gitleaks-baseline.json</code>
Checkov	<code>#</code> <code>checkov:skip=ID:reason</code>	<code>.checkov.yaml</code> <code>skip-</code> <code>check</code>	—
ClamAV	—	—	—

Running GitHub Actions locally

NOTE: This uses the `act` tool and assumes access to a valid AWS CodeBuild project `codebuild-project` in "us-east-1"

```
act --platform ubuntu-latest==self-hosted \
  --job build \
  --workflows .github/workflows/codebuild.yml \
  --env-file .env \
  --var CODEBUILD_PROJECT_NAME=codebuild-project \
  --var AWS_REGION=us-east-1 \
  --var ROLE_DURATION_SECONDS=7200 \
  --artifact-server-path=$PWD/.codebuild/artifacts \
  --cache-server-path=$PWD/.codebuild/artifacts \
  --env ACT_CODEBUILD_DIR=$PWD/.codebuild/downloads \
  --bind
```